Universidad _____

Facultad _____

Investigación de Operaciones

Calculadora del Método Simplex en Python

Prototipo 1.0

Catedrático _____

Integrantes del equipo

Nombre completo	Carné

Fecha de entrega _____

2 Introducción

La programación lineal permite elegir valores para un conjunto de variables con el propósito de maximizar un beneficio o minimizar un costo, respetando restricciones de recursos. Se aplica, por ejemplo, a la planificación de producción, la asignación de materiales y la distribución de presupuestos. El Método Simplex busca una solución óptima mediante cambios de base que conservan la factibilidad del modelo.

Este proyecto presenta un prototipo funcional del Simplex tabular en Python. El programa convierte restricciones de tipo menor o igual en ecuaciones, construye una tabla numérica y muestra cada pivoteo. Los dos ejemplos incorporados alcanzan los valores óptimos $Z = 36$ y $Z = 21$; la salida se contrasta con soluciones teóricas y con pruebas automatizadas.

3 Objetivos

Objetivo general

Desarrollar un programa informático en Python capaz de resolver modelos de programación lineal mediante el Método Simplex tabular, dentro del alcance establecido para el Prototipo 1.0.

Objetivos específicos

- Traducir la lógica matemática del pivoteo a estructuras de datos computacionales y operaciones elementales sobre filas.
- Validar los resultados del algoritmo con al menos dos ejercicios resueltos en clase, comparando sus tablas y soluciones con la resolución teórica.

Confirmación pendiente: se utilizan los dos ejemplos actuales del programa; falta confirmar su correspondencia con los ejercicios resueltos en clase.

4 Alcance y limitaciones del Prototipo 1.0

El prototipo resuelve maximización con variables continuas no negativas, restricciones de tipo $\leq$ y lados derechos no negativos. Admite una o más variables y restricciones, agrega holguras y muestra las tablas inicial, intermedias y final. Permite utilizar ejemplos definidos en el código o ingresar coeficientes por consola.

Se excluyen minimización, restricciones $\geq$ o $=$, lados derechos negativos, variables libres, variables artificiales, Gran M, Dos Fases e interfaz gráfica. Tampoco se resuelven modelos enteros o no lineales. La lectura de modelos desde archivos y el análisis de sensibilidad quedan para fases posteriores.

Los cálculos usan punto flotante. El programa detecta falta de acotación, limita las iteraciones y detiene una base repetida, pero no implementa una regla general anticiclaje ni técnicas avanzadas para modelos mal escalados.

5 Requisitos técnicos y entorno de desarrollo

El lenguaje utilizado es Python. La evidencia de este documento se ejecutó el 2 de octubre de 2026 en Windows 11 con las siguientes versiones. Las bibliotecas fijadas requieren Python 3.11 o posterior.

Componente	Versión	Función
Python	3.12.14	Control del algoritmo y entrada por consola
NumPy	2.3.5	Arreglos numéricos y operaciones matriciales
Pandas	3.0.1	Presentación de tablas en consola

El programa también se había verificado con Python 3.11.6. Las bibliotecas argparse, fractions, pathlib y unittest pertenecen a la biblioteca estándar; se utilizan para las opciones de ejecución, entrada de fracciones, archivos y pruebas.

Instalación y ejecución

Desde la carpeta del programa, instalar las dependencias en el mismo intérprete con el que se ejecutará el archivo:

```
python -m pip install numpy==2.3.5 pandas==3.0.1
python simplex.py
python simplex.py --interactivo
python simplex.py --reporte simplex_resultado.txt
python -m unittest -v test_simplex
```

La ejecución sin opciones resuelve los dos ejemplos. El modo interactivo solicita el número de variables, las restricciones y sus coeficientes; admite decimales y fracciones separados por espacios. La opción --reporte guarda la salida en un archivo .txt; --info muestra el alcance y las versiones.

Estructuras de datos principales

Para n variables de decisión y m restricciones, tabla es un arreglo bidimensional de NumPy de tamaño (m + 1) × (n + m + 1). Las primeras m filas contienen las restricciones y la última representa la función objetivo.

```
Restricciones:  [ A  | I | b ]
Fila objetivo:  [-c  | 0 | 0 ]
```

A almacena los coeficientes del modelo; I es la identidad de las variables de holgura y b es el lado derecho. La columna explícita de Z se omite porque su coeficiente es 1 en la fila objetivo y 0 en las restantes.

Las listas nombres_columnas y base relacionan las posiciones de la matriz con X1, X2, … y S1, S2, …. El resultado es un diccionario con Z y las variables de decisión. Pandas crea un DataFrame para imprimir la tabla; el pivoteo se calcula en NumPy.

6 Arquitectura y lógica del algoritmo

Conversión a forma estándar

A cada restricción se agrega una variable de holgura no negativa. Con b no negativo, X = 0 y S = b constituyen una solución básica factible inicial.

$$AX + S = b$$
$$Z - c^T X = 0$$

La fila objetivo contiene los coeficientes negados. Mientras exista un coeficiente menor que -10^{-9}, puede seleccionarse una variable de entrada.

Pseudocódigo del ciclo Simplex

```
INICIO
  Leer c, A y b
  Validar dimensiones, números finitos y b >= 0
  Agregar holguras y construir [A | I | b; -c | 0 | 0]
  Inicializar la base con S1, ..., Sm y mostrar tabla

  MIENTRAS exista un coeficiente de Z < -1e-9
    Si se supera el límite o se repite una base: detener
    Elegir la columna con coeficiente más negativo
    Calcular b_i / a_iq solo donde a_iq > 1e-9
    Si no hay fila elegible: informar "no acotado"
    Elegir la fila con el cociente mínimo
    Mostrar entrada, salida, razones y elemento pivote
    Dividir toda la fila pivote por el elemento pivote
    Anular la columna pivote en las demás filas, incluida Z
    Actualizar la base, contar iteración y mostrar tabla
  FIN MIENTRAS

  Extraer X y Z y verificar factibilidad y objetivo
  Si la verificación falla: informar error numérico
  Mostrar tabla final y devolver la solución óptima
FIN
```

Criterios de selección

La variable de entrada corresponde al coeficiente más negativo de la fila Z. La variable de salida se determina con el menor cociente entre el lado derecho y un coeficiente positivo de la columna pivote. El cociente cero es válido. En empates exactos se selecciona la primera columna o fila en el orden de la tabla.

Operaciones de Gauss Jordan

La fila pivote se divide entre su pivote para convertirlo en 1. Después se resta a cada otra fila el múltiplo de la fila normalizada que anula su entrada en esa columna. Las mismas operaciones se aplican a la fila Z y a la columna b.

```
T[p, :] = T[p, :] / T[p, q]
Para cada i distinto de p:
    factor = T[i, q]
    T[i, :] = T[i, :] - factor * T[p, :]
```

Funciones principales del programa

La clase SimplexTabular concentra los datos y las operaciones matemáticas. Las funciones de consola preparan los modelos, presentan la solución y conservan la evidencia de ejecución.

Función o método	Responsabilidad
_validar_datos()	Comprueba dimensiones, coeficientes finitos y b no negativo.
_construir_tabla_inicial()	Agrega la identidad de holguras y la fila objetivo.
es_optima()	Revisa la ausencia de coeficientes negativos significativos en Z.
encontrar_columna_pivote()	Selecciona el coeficiente más negativo de Z.
calcular_cocientes()	Obtiene razones y excluye denominadores no positivos.
encontrar_fila_pivote()	Selecciona la razón mínima o informa falta de acotación.
actualizar_tabla()	Normaliza la fila pivote y ejecuta Gauss-Jordan.
resolver()	Controla el ciclo, las paradas y la verificación final.
obtener_solucion()	Extrae Z y las variables; las no básicas valen cero.
verificar_solucion()	Comprueba $X \geq 0$, $AX \leq b$ y la consistencia del objetivo.
resolver_caso()	Muestra el modelo y contrasta el resultado con la referencia.
modo_interactivo() y main()	Gestionan la entrada, las opciones y el reporte de texto.

Control de errores y terminación

Una fila con coeficiente cero o negativo en la columna pivote se excluye de la razón mínima. El método de actualización también rechaza un pivote nulo o demasiado pequeño. Si no hay fila elegible, se informa que el problema es no acotado dentro de la tolerancia usada.

El límite predeterminado es de 1000 iteraciones por modelo y puede cambiarse con --max-iteraciones. El programa guarda las bases visitadas para detectar repeticiones y detener un posible ciclo. Estas paradas no se presentan como soluciones óptimas.

La tolerancia de selección y optimalidad es 10^{-9}. La comprobación final utiliza 10^{-7}, con ajuste por escala para las restricciones y tolerancias absoluta y relativa para el objetivo. La salida muestra hasta ocho cifras significativas; las operaciones conservan los valores en punto flotante.

7 Pruebas y validación

Se ejecutaron los dos ejemplos incorporados en simplex.py. Cada caso muestra la entrada, la tabla final y la comparación con una solución teórica. Está pendiente confirmar que correspondan a los ejercicios de clase.

Caso de prueba 1

$$\text{Maximizar } Z = 3X_1 + 5X_2$$
$$X_1 \leq 4$$
$$2X_2 \leq 12$$
$$3X_1 + 2X_2 \leq 18$$
$$X_1, X_2 \geq 0$$

Entrada definida en CASOS_PRUEBA y ejecutada con python simplex.py:

```
c = [3, 5]
A = [[1, 0], [0, 2], [3, 2]]
b = [4, 12, 18]
```

En la primera iteración entra X2 y sale S2, con pivote 2. En la segunda entra X1 y sale S3, con pivote 3.

```
TABLA FINAL
Base  X1  X2  S1          S2          S3  b
  S1   0   0   1  0.33333333 -0.33333333  2
  X2   0   1   0         0.5           0  6
  X1   1   0   0 -0.33333333  0.33333333  2
   Z   0   0   0         1.5           1 36
Criterio de parada: fila Z sin valores < -1e-09.

SOLUCION OPTIMA
  Z = 36
  X1 = 2
  X2 = 6
Iteraciones realizadas: 2

RESULTADO: VALIDACION CORRECTA
```

Salida 1. Extracto de la ejecución real de simplex.py.

Captura de pantalla de la ejecución pendiente de incorporar.

Interpretación. La solución es X1 = 2, X2 = 6 y Z = 36. Las holguras son S1 = 2, S2 = 0 y S3 = 0; las restricciones segunda y tercera se cumplen con igualdad.

Comprobación teórica. Se verifica $2 \leq 4$, $2(6) = 12$ y $3(2) + 2(6) = 18$. Al multiplicar la segunda restricción por 1.5 y sumar la tercera se obtiene $3X1 + 5X2 \leq 36$. La solución alcanza esa cota, por lo que es óptima.

Caso de prueba 2

$$\text{Maximizar } Z = 5X_1 + 4X_2$$
$$6X_1 + 4X_2 \leq 24$$
$$X_1 + 2X_2 \leq 6$$
$$X_1, X_2 \geq 0$$

Entrada definida en CASOS_PRUEBA y ejecutada con python simplex.py:

```
c = [5, 4]
A = [[6, 4], [1, 2]]
b = [24, 6]
```

En la primera iteración entra X1 y sale S1, con pivote 6. En la segunda entra X2 y sale S2, con pivote 4/3.

```
TABLA FINAL
Base  X1  X2     S1   S2   b
  X1   1   0   0.25 -0.5   3
  X2   0   1 -0.125 0.75 1.5
   Z   0   0   0.75  0.5  21
Criterio de parada: fila Z sin valores < -1e-09.

SOLUCION OPTIMA
  Z = 21
  X1 = 3
  X2 = 1.5
Iteraciones realizadas: 2

RESULTADO: VALIDACION CORRECTA
```

Salida 2. Extracto de la ejecución real de simplex.py.

Captura de pantalla de la ejecución pendiente de incorporar.

Interpretación. La solución es X1 = 3, X2 = 1.5 y Z = 21. Ambas holguras valen cero, por lo que las dos restricciones se cumplen con igualdad.

Comprobación teórica. Se verifica 6(3) + 4(1.5) = 24 y 3 + 2(1.5) = 6. Multiplicar la primera restricción por 0.75 y la segunda por 0.5, y sumarlas, da 5X1 + 4X2 ≤ 21. La solución alcanza esa cota y es óptima.

El resultado también coincide directamente con la función objetivo: 5(3) + 4(1.5) = 21. Las dos variables son no negativas.

Comparación de resultados y pruebas adicionales

Caso	X1	X2	Z teórico	Z obtenido	Iteraciones
1	2	6	36	36	2
2	3	1.5	21	21	2

Los dos casos aprobaron los controles de no negatividad, restricciones, valor de la función objetivo y coincidencia con la referencia. Las cotas obtenidas como combinaciones no negativas de las restricciones proporcionan una comprobación de optimalidad independiente de las tablas del algoritmo.

También se ejecutó test_simplex.py con unittest. Las 24 pruebas terminaron correctamente; incluyen la comparación de 20 modelos pequeños con enumeración independiente de vértices, además de los siguientes comportamientos:

- Rechazo de dimensiones inconsistentes, datos no numéricos, NaN, infinito y lados derechos negativos.
- Cocientes cero, empates, restricciones redundantes y soluciones óptimas en la tabla inicial.
- Detección de problemas no acotados, límite de iteraciones y repetición de una base en un caso de ciclo.
- Entrada interactiva con fracciones y decimales, reintentos de entrada y exportación de reportes.

La evidencia conserva los modelos, las tablas y la salida de las pruebas. Estos resultados verifican el alcance implementado; la validación específica con ejercicios de clase queda pendiente de su identificación.

8 Conclusiones de la Fase 1

El prototipo traduce el pivoteo del Simplex a operaciones sobre un arreglo bidimensional. La identidad de las holguras permite iniciar con una base factible para el alcance admitido, y las listas de nombres mantienen la relación entre los números de la tabla y las variables del modelo.

El programa muestra cada iteración y obtiene los resultados esperados de los dos ejemplos. La sustitución en las restricciones y las cotas superiores confirman Z = 36 y Z = 21. Las pruebas automatizadas aportan evidencia adicional sobre los controles de entrada y las condiciones de terminación.

Los desafíos técnicos del diseño fueron preservar la fila pivote normalizada, actualizar correctamente la base, excluir divisiones por cero y evitar ciclos indefinidos. Se atendieron con operaciones sobre una copia de la tabla, validaciones del pivote, tolerancias numéricas y controles de bases e iteraciones.

Se cumple el objetivo de implementar el Método Simplex tabular dentro de las limitaciones declaradas. Para dar por cumplido el objetivo de validación con ejercicios resueltos en clase, aún debe confirmarse que los modelos documentados sean los utilizados por el catedrático.

9 Trabajo futuro para la Fase 2

Las siguientes mejoras ampliarían el conjunto de modelos y facilitarían el uso del programa:

- Leer coeficientes desde archivos .txt o .csv con un formato definido y validación de errores.
- Aplicar una regla anticiclaje, como Bland, y ampliar el tratamiento de degeneración y empates.
- Implementar Dos Fases o Gran M para construir una base inicial cuando las holguras no sean suficientes.
- Incorporar minimización, restricciones ≥ o =, lados derechos negativos y transformaciones de variables.
- Mejorar el escalado numérico y detectar soluciones óptimas alternativas.
- Desarrollar una interfaz gráfica para introducir el modelo y consultar sus iteraciones.

10 Referencias y bibliografía

Cornell University, CS 6820. (2020, octubre). Lecture notes on the simplex method. Apuntes académicos sobre bases, pivoteo y el algoritmo Simplex.

Consultar los apuntes de Cornell

Python Software Foundation. (s. f.). Documentación de Python 3.11. Biblioteca estándar, lenguaje y herramientas de ejecución.

Documentación oficial de Python 3.11

NumPy Developers. (s. f.). numpy.ndarray. NumPy v2.3 Manual. Representación de arreglos numéricos multidimensionales.

Referencia oficial de numpy.ndarray

pandas development team. (s. f.). pandas.DataFrame. Referencia de la estructura tabular utilizada para presentar los resultados.

Referencia oficial de pandas.DataFrame

Fuentes técnicas consultadas el 2 de octubre de 2026. La versión instalada de cada biblioteca se registra en el apartado 5.

Material de clase para completar la validación

Referencia _____

_____